Computer Systems and Networks

Prof. Ramachandran, Prof. Forsyth

Extra Credit Project

Due: **April 28th 2026**

1 Project Overview and Description

The datapath design that we implemented for Project 1 (LC-5200) was, in fact, grossly inefficient. This is your chance to show that you can optimize the performance of LC-5200 using the concepts you learned in this class. Ranging from optimizing the datapath to building a pipeline, it is up to you how you want your "Little Computer" to look like! (Hint: It must be better than LC-5200). In the real world, that means higher performance, lower power draw, and most importantly, happy customers!

2 Project Requirements

Before beginning the project, make sure you have done the following:

- Download the proper version of CircuitSim. The proper version is 1.11.2-CE. In the Files tab on Canvas, follow the instructions from the installation guide to download, install, and run CircuitSim.
- CircuitSim is still under development and may have unknown bugs. Please back up your work using some form of version control, such as a local/private git repository or Dropbox. **Do not use public git repositories; it is against the Georgia Tech Honor Code**.
- The LC-5200b assembler is written in Python. If you do not have Python 3+ installed on your system, you must install it before you continue. You may also use the Docker container for this.

3 Grading

This project will be graded by submitting it to the **autograder on Gradescope**. The autograder is very bare-bones and tests only your full implementation of the processor. Your project only has to support the LC-5200 instructions and does not need to be able to handle interrupts.

Your processor will be graded on its speedup over the project 1 processor when executing three separate programs. Your implementation will be split into 2 categories depending on the number of RAM components on the circuit (**pipelined** submissions will use 2 RAM components, while **datapath** submissions will use 1). See 6 for more details.

Your grade will be in the range from 0 to 4 based on your speedup where 1x (no speedup) will not receive any credit and the maximal pipelined processor with data forwarding will receive 4 points. Non-pipelined implementations can receive up to 2 points. Anything in between will receive an interpolated grade, normalized to reflect the relative improvements to the original datapath. The autograder on Gradescope will reflect your final score (barring any honor code violations).

You will also be required to write a 2-3 page report discussing your processor implementation and the speedups you implemented. Please see Report for more details. The report will not be graded on its presentation, but it should be neatly typed and presentable. **Submissions without a report will result in no extra credit points**.

4 Project Objective

In this project, you will be creating a processor that implements a speedup over the single datapath processor that was implemented in project 1. You can implement any optimizations over the datapath created in project 1 necessary to reduce the number of clock cycles it takes to run any given program.

Here are some ideas to begin optimizing your processor:

- Use a Dual Ported Register File (DPRF)
- Use a Dual-Bus Design
- Create a 5-stage pipeline (using busy bit or data forwarding)

Your processor should fall into one of two categories: datapath or pipeline. A datapath design would resemble the processor you constructed for project 1, with 1 RAM and 1 microcontroller. A pipeline design should resemble the pipelined processor we discussed in class, with multiple stages and 2 RAMs. **Your implementation should fall into one of these two categories, or it may not pass the autograder**. For more information about submitting, see Deliverables.

Note: Your processor will implement the LC-5200b ISA. Before you move on, read Appendix A: LC-5200b Instruction Set Architecture to understand the ISA that you will be implementing. This is largely the same as the LC-5200 ISA, with some instructions omitted.

4.1 Datapath

If you implement a datapath design (e.g. DPRF, Dual-Bus), your circuit should contain **exactly 1 RAM**, among other components necessary to make the processor work. If you decide to take this approach, feel free to start with the processor you created in project 1. You may need to change some of the microcode associated with the project. If you want to create completely new microcode, we also provide `microcode.xlsx`, which has some templates you can use for your microcode. For more information, see Appendix C: Microcode Template.

All of the scores are restricted to be between 0 and 2 points. For reference, different implementations from previous semesters achieved the following speedups:

- No improvements: 1.0x speedup, 0 points
- Outputting immediate value directly into ALU: 1.09x speedup, 0.3 points
- Dual-ported register file: 1.5x speedup, 0.99 points

Note that these speedups may not be accurate depending on the number and the level of optimization you perform on your processor.

4.2 Pipeline

If you implement a pipeline design, your circuit should contain **exactly 2 RAMs** (D-MEM & I-MEM), as you would see in Harvard architecture, among other components necessary to make the pipeline work. **Your pipeline must also have 5 stages**. Since this is the first time you will be implementing a pipelined processor in CircuitSim, we provide some additional tips in Appendix B: Pipelined Processor Guide.

All of the scores are restricted to be between 0 and 4 points. For reference, different implementations from previous semesters achieved the following speedups:

- Pipelined processor with busy bit
- Pipelined processor with data forwarding: 4.5x-4.8x speedup, 4 points
- Pipelined processor with data forwarding and backward-taken: 5x-6x speedup, 4 points

Note that these speedups may not be accurate depending on the number and the level of optimization you perform on your processor.

4.3 Report

Alongside the project, you will be required to submit a written report, roughly 2-3 pages in length. The report should be presentable, with appropriate formatting.

Contents of the report may include, but are not limited to:

- Explanation of how to load your ROM(s) with your microcode.
- An explanation of your implementation along with why you chose that implementation.
- Challenges that were faced during development.
- Results relating to cycle count when running the pow.s file, and any associated pipelining metrics that were taught in class.
- Potential areas of improvement or further optimization.

Submissions without a report will result in no extra credit points.

5 Provided Resources

For your processor, you will be provided the following files:

- `LC-5200b.sim`: Empty template CircuitSim file.
- `microcode.xlsx`: Template microcode file. For more information, see Appendix C: Microcode Template
- `pow.s`: Assembly script to test your processor with.

6 Deliverables

You may receive **up to 4 points of extra credit on your final grade** by completing this project.

- Submissions without an adequate report will receive no credit for the project.
- Partial credit may or may not be awarded for effort given to a pipeline with minor errors.
- Due to the project being due on the last instructional day of the semester, you cannot use late days or submit regrades for this project.

To submit your project, you need to upload the following files to Gradescope:

- LC-5200b.sim
- microcode.xlsx (if applicable, but not required)
- report.pdf

Your CircuitSim file (LC-5200b.sim) should satisfy the following requirements:

- Your main datapath should be in the circuit labeled `Datapath`.
- All of the microcode should be pre-loaded into the ROMs. You still need to submit the microcode Excel file, if applicable.
- Reference registers in the register file in lower case with a '$' prefix ($zero, $at, $v0...), according to Appendix A: LC-5200b Instruction Set Architecture.
- If you implement a datapath design:

  - Your PC register should be labeled `PC`.
  - Your memory (RAM) should be labeled `RAM`.
- If you implement a pipeline:
  - The PC register in your `Fetch` stage should be labeled `PC-IF`.
  - Your instruction memory (RAM for instructions only) should be labeled `I-MEM`, and your data memory (RAM for instructions and modified data) should be labeled `D-MEM`.

Always re-download your assignment from Gradescope after submitting to ensure that all necessary files were properly uploaded. If what we download does not work, you will get a 0 regardless of what is on your machine.

7 Appendix A: LC-5200b Instruction Set Architecture

The LC-5200b is a simple, yet capable computer architecture. The LC-5200b ISA is based off the LC-2200 ISA defined in the Ramachandran & Leahy textbook for CS 2200 with a few additional instructions. We highly recommend reading the textbook to understand how an ISA works. It may also provide further hints on how to implement some of the instructions

This ISA is the same as the one you implemented in project 1, without the `MIN` and `MAX` instructions. This is also the same as the LC-2200 present in the textbook with the addition of the `BGT` and `LEA` instructions.

The LC-5200b is a **word-addressable**, **32-bit** computer. **All addresses refer to words**, i.e. the first word (four bytes) in memory occupies address 0x0, the second word, 0x1, etc.

All memory addresses are truncated to 16 bits on access, discarding the 16 most significant bits if the address was stored in a 32-bit register. This provides roughly 262 KB of addressable memory.

7.1 Registers

The LC-5200b has 16 general-purpose registers. While there are no hardware-enforced restraints on the uses of these registers, your code is expected to follow the conventions outlined below.

Table 1: Registers and their Uses

Register Number	Name	Use	Callee Save?
0	$zero	Always Zero	NA
1	$at	Assembler/Target Address	NA
2	$v0	Return Value	No
3	$a0	Argument 1	No
4	$a1	Argument 2	No
5	$a2	Argument 3	No
6	$t0	Temporary Variable	No
7	$t1	Temporary Variable	No
8	$t2	Temporary Variable	No
9	$s0	Saved Register	Yes
10	$s1	Saved Register	Yes
11	$s2	Saved Register	Yes
12	$k0	Reserved for OS and Traps	NA
13	$sp	Stack Pointer	No
14	$fp	Frame Pointer	Yes
15	$ra	Return Address	No

1. **Register 0** is always read as zero. Any values written to it are discarded.
   Note: For this project, you must implement the zero register. Regardless of what is written to this register, it should always output zero.
2. **Register 1** is used to hold the target address of a jump. Pseudo-instructions generated by the assembler may also be used.
3. **Register 2** is where you should store any returned value from a subroutine call. A quick way to check if **pow.s** runs correctly to verify if this register's value is correct.
4. **Registers 3 - 5** are used to store function/subroutine arguments.
   Note: Registers 2 through 8 should be placed on the stack if the caller wants to retain those values. These registers are fair game for the callee (subroutine) to trash.
5. **Registers 6 - 8** are designated for temporary variables. The caller must save these registers if they want these values to be retained.

6. **Registers 9 - 11** are saved registers. The caller may assume that the subroutine never tampered with these registers. If the subroutine needs these registers, then it should place them on the stack and restore them before they jump back to the caller.
7. **Register 12** is reserved for handling interrupts. While it should be implemented, it otherwise will not have any special use on this assignment.
8. **Register 13** is the everchanging top of the stack; it keeps track of the top of the activation record for a subroutine.
9. **Register 14** is the anchor point of the activation frame. It is used to point to the first address on the activation record for the currently executing process.
10. **Register 15** is used to store the address a subroutine should return to when it is finished executing.

7.2 Instruction Overview

The LC-5200b supports a variety of instruction forms, only a few of which we will use for this project. The instructions we will implement in this project are summarized below.

Table 2: LC-5200b Instruction Set

	31 30 29 28	27 26 25 24	23 22 21 20	19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4	3 2 1 0
ADD	0000	DR	SR1	unused	SR2
NAND	0001	DR	SR1	unused	SR2
ADDI	0010	DR	SR1	immval20	
LW	0011	DR	BaseR	offset20	
SW	0100	SR	BaseR	offset20	
BEQ	0101	SR1	SR2	offset20	
JALR	0110	AT	RA	unused	
HALT	0111	unused			
BGT	1000	SR1	SR2	offset20	
LEA	1001	DR	unused	offset20	

7.3 Detailed Instruction Reference

7.3.1 ADD

Assembler Syntax

```
ADD    DR, SR1, SR2
```

Encoding

31 30 29 28	27 26 25 24	23 22 21 20	19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0	
0000	DR	SR1	unused	SR2

Operation

```
DR = SR1 + SR2;
```

Description

The ADD instruction obtains the first source operand from the SR1 register. The second source operand is obtained from the SR2 register. The second operand is added to the first source operand, and the result is stored in DR.

7.3.2 NAND

Assembler Syntax

```
NAND   DR, SR1, SR2
```

Encoding

31 30 29 28	27 26 25 24	23 22 21 20	19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0	
0001	DR	SR1	unused	SR2

Operation

```
DR = ~(SR1 & SR2);
```

Description

The NAND instruction performs a logical NAND (AND NOT) on the source operands obtained from SR1 and SR2. The result is stored in DR.

> **HINT:** A logical NOT can be achieved by performing a NAND with both source operands the same. The following assembly:
>
> `NAND DR, SR1, SR1`
>
> achieves the following logical operation: $DR \leftarrow \overline{SR1}$.

7.3.3 ADDI

Assembler Syntax

```
ADDI   DR, SR1, immval20
```

Encoding

31 30 29 28	27 26 25 24	23 22 21 20	19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
0010	DR	SR1	immval20

Operation

```
DR = SR1 + SEXT(immval20);
```

Description

The ADDI instruction obtains the first source operand from the SR1 register. The second source operand is obtained by sign-extending the immval20 field to 32 bits. The resulting operand is added to the first source operand, and the result is stored in DR.

7.3.4 LW

Assembler Syntax

```
LW    DR, offset20(BaseR)
```

Encoding

31 30 29 28	27 26 25 24	23 22 21 20	19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
0011	DR	BaseR	offset20

Operation

```
DR = MEM[BaseR + SEXT(offset20)];
```

Description

An address is computed by sign-extending bits [19:0] to 32 bits and then adding this result to the contents of the register specified by bits [23:20]. The 32-bit word at this address is loaded into DR.

7.3.5 SW

Assembler Syntax

```
SW    SR, offset20(BaseR)
```

Encoding

31 30 29 28	27 26 25 24	23 22 21 20	19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
0100	SR	BaseR	offset20

Operation

```
MEM[BaseR + SEXT(offset20)] = SR;
```

Description

An address is computed by sign-extending bits [19:0] to 32 bits and then adding this result to the contents of the register specified by bits [23:20]. The 32-bit word obtained from register SR is then stored at this address.

7.3.6 BEQ

Assembler Syntax

```
BEQ  SR1, SR2, offset20
```

Encoding

31 30 29 28	27 26 25 24	23 22 21 20	19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
0101	SR1	SR2	offset20

Operation

```
if (SR1 == SR2) {
    PC = incrementedPC + offset20
}
```

Description

A branch is taken if SR1 is equal to SR2. If this is the case, the PC will be set to the sum of the incremented PC (since we have already undergone fetch) and the sign-extended offset[19:0].

7.3.7 JALR

Assembler Syntax

```
JALR   AT, RA
```

Encoding

31 30 29 28	27 26 25 24	23 22 21 20	19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
0110	AT	RA	unused

Operation

```
RA = PC;
PC = AT;
```

Description

First, the incremented PC (address of the instruction + 1) is stored in register RA. Next, the PC is loaded with the value of register AT, and the computer resumes execution at the new PC.

7.3.8 HALT

Assembler Syntax

```
HALT
```

Encoding

31 30 29 28	27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
0111	unused

Description

The machine is brought to a halt and executes no further instructions.

> **Note:** The autograder will not run if this instruction is not implemented correctly.

7.3.9 BGT

Assembler Syntax

```
BGT  SR1, SR2, offset20
```

Encoding

31 30 29 28	27 26 25 24	23 22 21 20	19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
1000	SR1	SR2	offset20

Operation

```
if (SR1 > SR2) {
    PC = incrementedPC + offset20
}
```

Description

A branch is taken if SR1 is greater than SR2. If this is the case, the PC will be set to the sum of the incremented PC (since we have already undergone fetch) and the sign-extended offset[19:0].

7.3.10 LEA

Assembler Syntax

```
LEA    DR, label
```

Encoding

31 30 29 28	27 26 25 24	23 22 21 20	19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
1001	DR	unused	PCoffset20

Operation

```
DR = PC + SEXT(PCoffset20);
```

Description

An address is computed by sign-extending bits [19:0] to 32 bits and adding this result to the incremented PC (address of instruction + 1). It then stores the computed address in the register DR.

8 Appendix B: Pipelined Processor Guide

Since you have not implemented a pipelined processor before, we provide some additional hints to get you started on your implementation. Building a pipelined processor will generally result in the highest amount of speedup.

8.1 Building the Pipeline

First, you will have to build the hardware to support all of your instructions. You will have to make each stage such that it can accommodate the actions of all instructions passing through it. Use the book (Ch. 5) to get an idea of what the pipeline looks like and to understand the function of each stage before you start building your circuits.

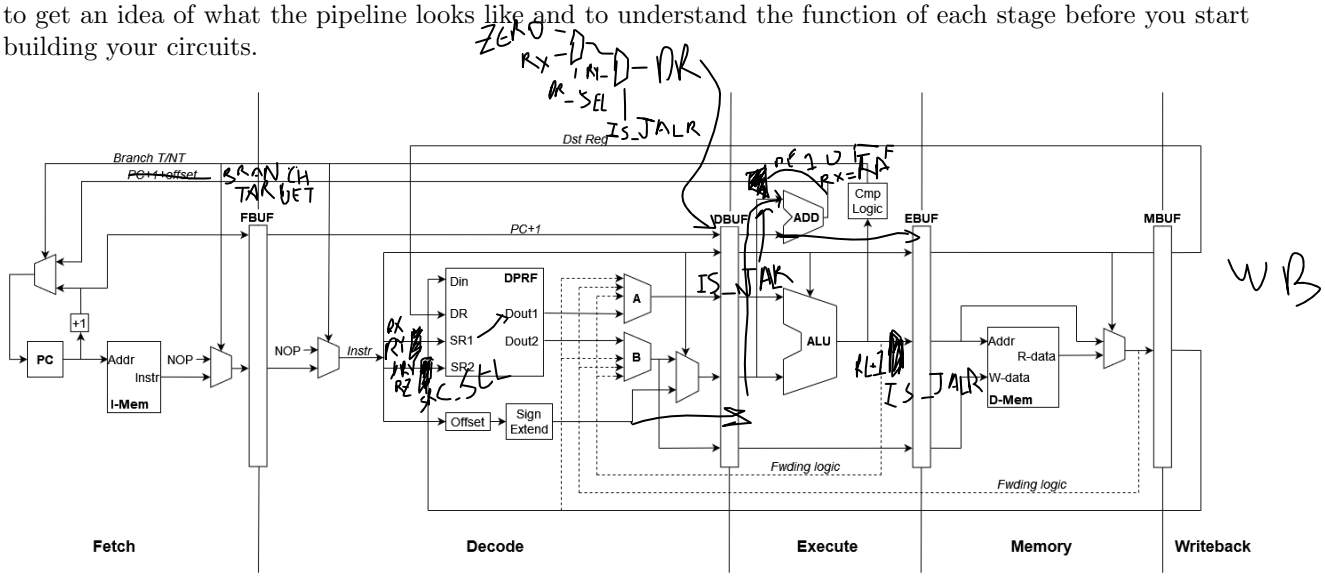

Figure 1: Pipeline Schematic

8.1.1 IF Stage

Functionality:

- **PC Update**: In the IF stage, we need to update the PC's value in order to fetch the proper next instruction. For normal sequential execution, the IF stage should update the PC by incrementing it by 1. However, in the case of branches such as with JALR and BEQ, the PC's value should be set to whatever new address was calculated during the EX Stage.
- **Fetch from I-MEM**: We will then use the PC value to index into I-MEM and retrieve an instruction. Note that I-MEM has 16 address bits, so you will need some circuit to reduce the PC's bits from 32 bits to 16 bits.
- **Selecting the next PC**: Choosing whether the PC should be updated to PC+1 or to the address calculated in EX can be accomplished with a multiplexer. Consider what the selector bits for this MUX must be; they come from the EX Stage.

8.1.2 ID/RR Stage

Functionality:

- **Obtaining Data from the Instruction**: The first thing to accomplish in the ID/RR stage is to obtain the opcode of the 32 bit instruction that was fetched in the IF stage. Consider all other values that need to be obtained directly from the instruction, such as register numbers and the immediate/offset value.

- **Decoding the Instruction**: Once we have the opcode for our instruction, we need to determine the specific control signals that need to be asserted. Like our LC-5200 uniprocessor that we implemented in projects 1 and 2, we can use ROMs that store microcode for each instruction's control signals. Given the opcode, the ROMs should output the necessary control signals required for a specific instruction.
- **Reading Registers**: Our instruction might require that we read from one or two registers. If reading from a single register, we use only 1 port in our dual-ported register file (DPRF), using the register number specified in the instruction to read a value. If given two register numbers, then we utilize both ports in the DPRF to read from both simultaneously.
- **Selecting "A" and "B"**: Like the ALU in projects 1 and 2, the EX Stage utilizes two operands, "A" and "B". You must select the proper value for A and the proper value for B based on the instruction being executed (for example, select PCOffset20, immval20, a register value, etc.).

Considerations for Implementation:

- **ROMs**: Unlike the processor implemented in projects 1 and 2, there is no need for next state bits. Implement a component using one or more ROMs that takes in an opcode and returns the control signals associated with that opcode.
- **Dual Ported Register File**: When implementing the DPRF, consider that you only need to read from two registers at once. You will not have to write to more than one register simultaneously.
- **Data Forwarding**: If you decide to implement data forwarding, note that your selector for A and B should also be able to select any forwarded values.

8.1.3 EX Stage

Functionality:

- **Calculation**: The EX Stage must compute calculations using A and B. For example, it will calculate the sum for an ADD instruction, or the Base + Offset computation for LW and SW. This should be done with a complete ALU, capable of performing adding, nanding, and any other required operation.
- **Branch Evaluation**: The EX Stage must evaluate a branch condition, and then determine if the branch is taken or not taken. This can be performed by installing a comparison logic unit.
- **Branching**: As previously mentioned, the IF stage will require information about the branch evaluation performed in the EX Stage. Consider implementing forwarding lines that can forward this information between stages.

8.1.4 MEM Stage

Functionality:

- **Address Calculation**: The effective address for memory operations is derived from the Execute (EX) stage, typically involving arithmetic operations or immediate values combined with base register values. In systems with a 16-bit address space, it is crucial to use only the lower 16 bits of the calculated address, requiring a masking operation to discard any higher-order bits.
- **Read Operation**: Load instructions necessitate reading data from the memory address calculated in the MEM stage. This involves initiating a memory access with the calculated address and retrieving the corresponding data.
- **Write Operation**: Store instructions require writing data from a source register to the memory address determined in the MEM stage. The operation must ensure data is correctly written to the intended memory location.

8.1.5 WB Stage

Functionality:

- The WB stage selectively writes values back to the registers. This involves interfacing directly with the data in and write enable inputs of the DPRF, ensuring that results of computations or memory operations are correctly stored.
- The dual-ported nature of the DPRF allows simultaneous read and write operations on different registers within the same clock cycle. This design facilitates sharing of the DPRF between the ID/RR and WB stages.
- Control Logic: Implementing control logic within the WB stage is crucial for determining whether a write-back operation is required based on the instruction type.
- Data Selection: The WB stage must select the correct data source for write-back operations. This is typically between data fetched from memory or computation results generated in previous stages. Multiplexers, guided by control signals, can be a good design choice.

8.2 General Advice

8.2.1 Subcircuits

For this project, we highly encourage using modular design and creating subcircuits when necessary. We **strongly** recommend using subcircuits when building your pipeline buffers, stages, and forwarding unit. A modular design will make it easier to debug and test your circuit during development.

8.2.2 Pipeline Buffers

- Identify and support the requirements of all possible instructions by analyzing their needs.
- Pass a union of all requirements through the buffers to ensure functionality across diverse instructions.
- Optionally, implement dynamic buffer space utilization to optimize for instruction-specific requirements.

8.2.3 Control Signals

In the Project 1 datapath, recall that we had one main ROM that was the single source of all the control signals on the datapath. Now that we are spreading out our work across different stages of the pipeline, you have a choice of how to implement your signals!

The first thing to note is that in a pipelined processor, each stage is like a simple one-cycle processor that can do exactly ONE thing intended for that stage in a single cycle. In this sense, there is really no need for a control ROM anymore! Therefore, in real processors, each stage of the pipeline is implemented using hardwired control as discussed in Chapter 3 of the textbook. However, to keep your design simple for debugging and getting it working, we are going to suggest using a control ROM to generate the needed control signals for the different independent stages of the pipelined processor.

There are a few options for how to implement control signals:

1. Implement a single large main ROM in ID/RR which calculates all the control signals for every stage.
2. Implement small(er) ROMs in each stage that takes in the opcode and assert the proper signals for that operation.
3. Use hardwired combinational logic to obtain control signals without ROMs altogether.

8.2.4 Data Hazards

One must stall the pipeline when an instruction cannot proceed to the next stage because a value is not yet available to an instruction. This usually happens because of a data hazard. For example, consider two instructions in the following program:

1. `LW $t0, 5($t1)`
2. `ADDI $t0, $t0, 1`

Without stalling the ADDI instruction in the ID/RR stage, it will get an out of date value for $t0 from the regfile, as the correct value for $t0 isn't known the LW reaches the MEM stage! Therefore, we must stall. Consult the textbook (or your notes) for more information on data hazards.

To stall the pipeline, the stages preceding the stalled stage should disable writes into their buffers, i.e. they should continue to output the previous value into the next stage. The stalled stage itself will output NOOP (example, ADD $zero, $zero, $zero) instructions down the pipeline until the cause of the stall finishes.

Like we discussed in class, the two main ways to mitigate data hazards are using a **busy bit** or **data forwarding**. Either solution will work, but you may notice that you can eliminate a good deal of stalls by implementing data forwarding. This allows the ID/RR stage to retrieve values computed in later stages of the pipeline early so that stalling the instruction is not necessary. It is strongly recommended that you not use the busy bit/read pending bit strategy suggested in the book - this has some very nasty edge cases and requires much more logic than necessary.

If you decide to implement data forwarding, it is recommended that you make a forwarding unit that implements various stock rules. The forwarding unit should take in the two register values you are reading, the output value from the EX stage, the output value from the MEM stage, and the output value from the WB stage. To forward a value from a future stage back to ID/RR, you must check to see if the destination register number from a particular stage is equal to your source register numbers in the ID/RR stage. If so, you must forward the value from that stage to your ID/RR stage.

Note that forwarding cannot save you from one situation: when the destination register of an LW instruction is the source register of an instruction immediately after it. In this case, sometimes called "load-to-use", you must stall the instruction in the ID/RR stage.

Stalling

- Initiate stalling when data hazards prevent instruction progression to maintain data integrity.
- Implement stalling by disabling buffer writes in the preceding stages and issuing NOP instructions until the hazard is resolved.

Data Forwarding

- Use data forwarding to minimize stalls by allowing early access to values computed at later stages.
- Design a forwarding unit that evaluates the necessity for forwarding based on register comparisons.
- Address limitations, acknowledging scenarios like 'load-to-use' hazards that still require stalling.
- The priority encoder is a hardware component in CircuitSim that can choose priority between stages, resolving all WAW hazards without additional bubbles.

Keep in mind: the zero register can never change, therefore, it should not be considered for forwarding and stalling situations.

8.2.5 Control Hazards

Since branch instructions (BEQ/BGT/JALR) are resolved in EX, the pipeline may be unsure of which instructions are correct to fetch. This issue is an example of a control hazard that we discussed in class. The easiest ways to resolve control hazards are through **conservative branching**, which just stalls until you know the outcome of the branch, or **branch prediction**.

The easiest way to handle control hazards is conservative branching, which you can implement by using the same stalling logic you use for data hazards but only for the IF stage. However, this may be inefficient because you have to introduce 1-2 bubbles for each branch.

Although branch prediction may seem complex, a simple version of branch prediction to implement that is never worse than conservative branching is to predict the branch is always not taken. This works by just incrementing the PC as normal, but squashing the instructions in the IF and ID/RR stages if a branch is taken, and using the updated PC value.

Either solution is valid and will produce a pipeline with significant speedup over the sequential datapath we implemented in our first project. However, since branch prediction is a bit more complex, we have some notes/tips below to assist you should you choose to implement branch prediction.

Note: You can technically resolve JALR earlier than the EX stage. If you want to speed up your processor more, that is another point of improvement that you can make!

Always Not Taken Branch Prediction

- Always predict branch-not-taken, and flush the IF and ID/RR stages when a branch is taken.
- Address control hazards by predicting branch outcomes, with a default prediction that the branch is not taken.
- Implement hardware mechanisms to handle both correct predictions (continue normally) and incorrect predictions (flush incorrectly fetched instructions).
- Forward signals between stages (as needed) to alter the course of execution upon a branch.
- Develop a mechanism to flush the preceding stages upon a branch. **Hint:** What hardware component can we use to send a NOP into the FBUF and DBUF instead of the instruction that is squashed.
- Avoid using the "clear" port on registers to prevent timing/clock issues which operate asynchronously.

8.3 Testing

When you have constructed your pipeline, you should test it instruction by instruction to see if you have all the necessary components to ensure proper execution similar to how you did in Projects 1 and 2.

The best way to test if your pipeline is working successfully is to test it locally and verify that the final state of the registers is as expected. We recommend you write single-instruction test cases and pairs of instructions that result in common hazards such as RAW and Control hazards to test your logic. Once you've observed these test cases working, you should run `pow.s` to completion to verify that everything is working.

Be careful to only use the instructions listed in the appendix - there are some subtle points in having a separate instruction and data memory. Load the assembled program into both the instruction memory and the data memory and let your processor execute it. Any writes to memory will only affect the data memory.

9 Appendix C: Microcode Template

For your project, you will be provided an Excel template file for your microcode. Depending on your processor design, you may or may not need this file. Below are some tips to understanding this template and how you can use it. You can also feel free to make your own microcode Excel sheet. You can copy these templates to be used as many times as needed, or you can figure out how the formulas work for converting the columns into hex and create your own microcode. Again, this is just supposed to be used as a template and a starting point for your microcode. We have no restrictions on how you should make your microcode.

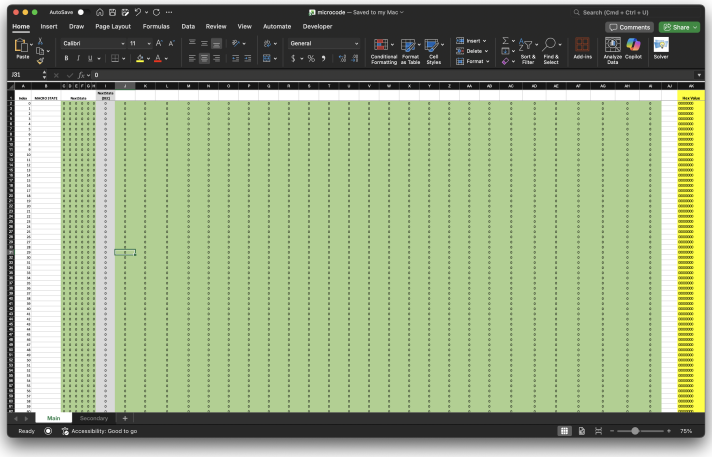

Figure 2: Main Microcode Template

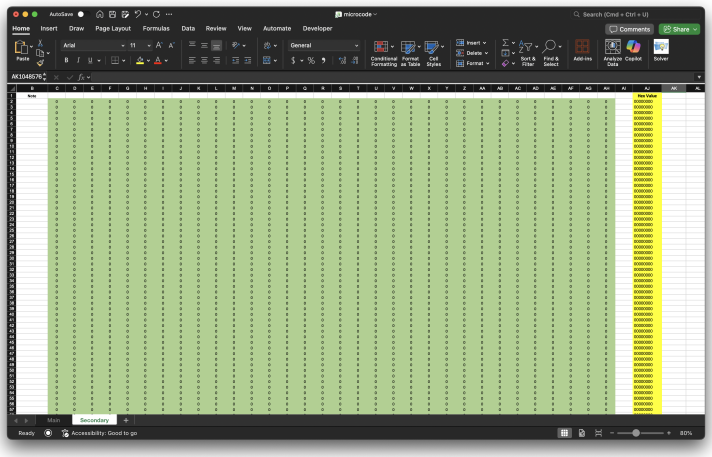

Figure 3: Secondary Microcode Template

9.1 Main Microcode

The main microcode is essentially an empty version of the microcode that you put into the Main ROM in projects 1 and 2. This is in the sheet labeled "Main", which can be selected at the bottom of the window. This is useful if you have a similar ROM in your project, where the next state is determined by the current state. This microcode template has a set of next state bits. **Note that the `NextState` bits displayed correspond to bits `[5:0]` in the ROM from left to right, while the remaining bits correspond to bits `[31:6]` from left to right**. The formulas are already set up to convert your microcode into hex to be able to be imported into your ROMs. This effectively means if your microcode uses the leftmost columns aside from the `NextState` bits, your ROM should have a bitsize of 32 bits. The columns that you do not use you can ignore.

9.2 Secondary Microcode

The secondary microcode template is for any ROMs that do not have the `NextState` bits. It is effectively the same template as the main template but doesn't have the `NextState` bits. This is provided in the sheet labeled "Secondary". **Note that the columns are assigned to bits `[31:0]`, from left to right. This means the leftmost column is the most significant bit and is bit 31 in your ROM.** Like with the main template, if you put your microcode in the leftmost columns, your ROM should have a bitsize of 32 bits.